

Can this backup continue the agent's history?

MemoryLineage checks whether a restored private snapshot is the authorized continuation of recorded history.

Verify the history,
not the memory.

ONE PRIVATE MEMORY · AN ORDERED HISTORY



RESTORED
State 1

Should the agent resume from here?

01 · THE PROBLEM

The file opens. The history may be wrong.

After an outage, restoring an old snapshot can silently rewind an agent while newer committed states already exist.

01

Snapshot 1

A private checkpoint is saved.

02

Snapshot 2

The recorded history advances.

03

Snapshot 3

The shared head is now here.

RESTORE

Snapshot 1

The operator brings an older copy back.

Restoring storage is easy.

The runtime can reopen a local backup.

Approving a continuation is different.

A controller needs independent evidence that this state may extend the recorded history.

Private snapshots. Shared order. Reviewable decisions.

Can this backup continue from the latest approved state?

PRIVATE SIDE · AGENT STORAGE

Keep the memory private

Prompts, notes, and raw memory values stay with the agent's own storage.



SHARED SIDE · EVIDENCE

Record a snapshot fingerprint

A fixed-size fingerprint shows a snapshot's place in history; ordered records show who approved each step without sharing raw memory.

MemoryLineage checks recorded order and authorization. It does not decide whether the memory itself is true or safe.

02 · THE SOLUTION · ARCHITECTURE

How a private backup becomes reviewable evidence.

Raw snapshots stay in private storage. The shared record keeps small fingerprints, order, and approval evidence.

WRITE PATH · CREATE THE SHARED HISTORY AND EVIDENCE

01 · PRIVATE DATA

Agent memory store

Raw snapshot stays local.

02 · EVIDENCE BUILDER

Rust evidence builder

Creates snapshot fingerprints and transition records.

RUST PRODUCES TWO SEPARATE OUTPUTS

SHARED CHECKPOINT

Solidity registry

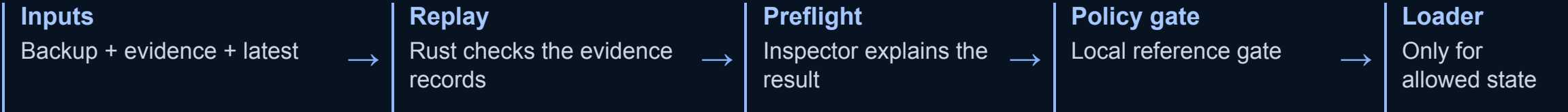
Latest state, previous state, approval.

PORTABLE EVIDENCE

Replayable bundle

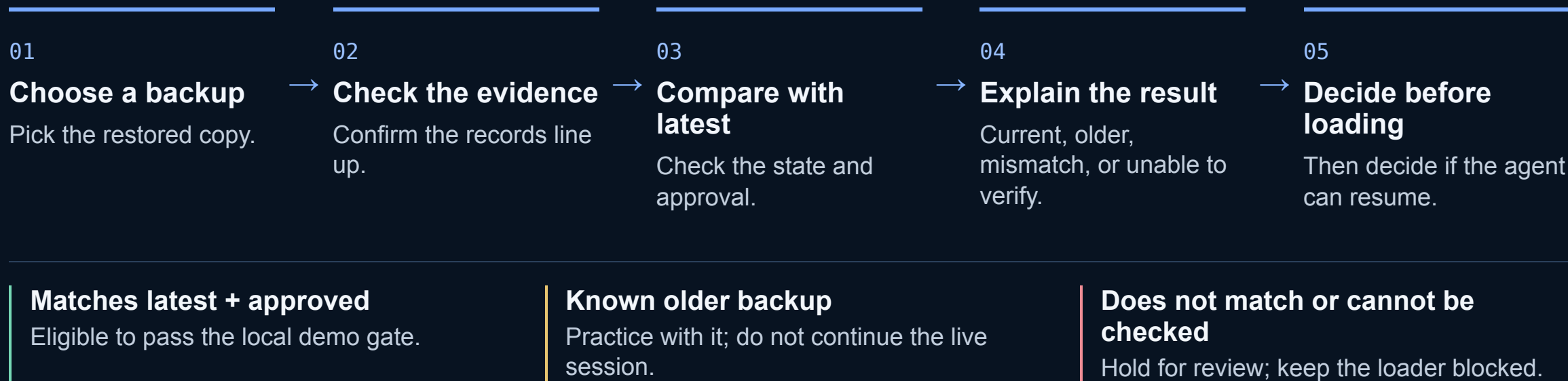
Fingerprints, transitions, signature details.

RESTORE PATH · COMPARE A RETURNED SNAPSHOT BEFORE LOADING



Demo Space V2 is synthetic and local: the checked-in Solidity bytecode runs through Rust/revm. This is not a public deployment.

Check the backup before the agent continues.



The protected loader is a framework-neutral reference adapter. External production runtime enforcement is not demonstrated.

The check works. The old backup stays on hold.

The latest recorded state is 3. The restored backup is state 1 and tries to continue from there.

STATE 1 → LATEST 3 / BACKUP 1

BAD_PREVIOUS_STATE

The local registry rejects a new step that starts from state 1 instead of state 3.

Recovery decision: known older backup → practice only → held before the local reference loader.

CHECK PASSED means the check caught the mismatch. The backup was not approved.

The screenshot displays the MEMORYLINEAGE interface, specifically the 'Tampering Lab' section. The top navigation bar includes 'Inspect', 'History', 'Tampering Lab', and 'Verify'. The main header shows 'TAMPERING LAB / FALSIFICATION WORKSPACE' and a breadcrumb trail: 'PUBLISHED LOCAL REVМ / RUST-WASH VERIFIED / DEMO-SPACE-V2-SILENT-ROLLBACK'.

The 'Tampering Lab' section has a subtitle: 'Restore an older snapshot, then try to continue the newer history. The result explains exactly what the registry accepts or rejects.'

The workflow is shown as a sequence of steps: 01 Restore (SNAPSHOT 1) → 02 Compare (STALE ROOT / HEAD) → 03 Replay (LOCAL RUST/REVM) → 04 Verdict (HOLD OLD BACKUP).

The 'CHOOSE A TEST' section lists several options, with 'LOCAL Silent Rollback EVIDENCE' selected. The 'WHAT WE ARE COMPARING' section shows a comparison between 'SYNTHETIC FIXTURE / CANONICAL SNAPSHOT' and 'LOCAL RESTORE ATTEMPT'. The synthetic fixture includes 'SNAPSHOT 1' (Language preference recorded), 'SNAPSHOT 2' (Evidence-backed claim policy recorded), and 'SNAPSHOT 3 / COMMITTED' (Raw-memory privacy boundary recorded, protocol root 0xa89ff09763...666f2f0). The local restore attempt includes 'RESTORE SNAPSHOT 1' (Language preference recorded) and 'ATTEMPT TRANSITION 4' (stale predecessor: 0x3635645aa8_d14f0ee6).

A green 'CHECK PASSED' button is visible. The 'Verdict' section shows a green checkmark and the text: 'Backup 1 cannot continue the history at Snapshot 3. The evidence check passed. The simulated continuation was rejected because Backup 1 is older than the current shared head. The exact machine reason remains in the evidence detail above.' Below this are buttons for 'Reset rehearsal' and 'Probe separate Sepolia'.

The 'WHAT THIS PROVES' section lists three points: 'Stale predecessor rejected' (The attempted root is not the canonical head), 'Canonical succession enforced' (Sequence and parent state are checked together), and 'Raw memory stays private' (Only commitments and metadata enter the evidence).

Why this recovery workflow is different.

The contribution is the recovery workflow around existing standards—not a new cryptographic primitive.

- Restore only

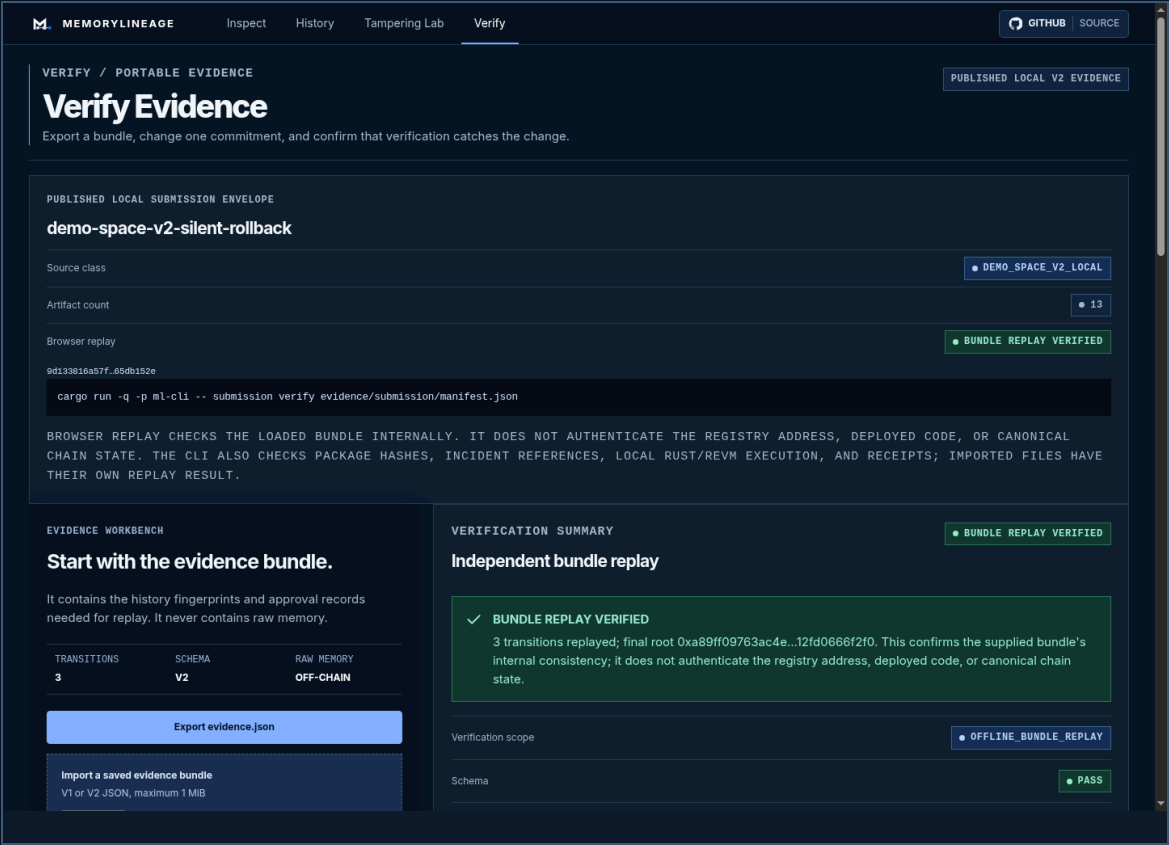
Reopens the file, but does not show whether it is the latest shared state.
- Local signed log

Shows who signed a record; by itself, it may not give other parties an independent latest checkpoint.

MemoryLineage Compares the backup with ordered shared evidence, then records a reviewable decision before the local loader.

original bundle → BUNDLE REPLAY VERIFIED
changed fingerprint → TRANSITION_ID_MISMATCH

Replay checks bundle consistency. This local demo does not authenticate public-chain origin.



Make recovery decisions reviewable.

When storage and resume approval are split across people, each needs the same evidence.

OPERATOR

See what came back

Compare the restored backup with the recorded latest state before handing control back.

CONTROLLER

Apply one visible rule

Latest and approved may pass the demo gate; older stays practice-only; unknown states are held.

AUDITOR

Check the same evidence

Replay the bundle independently and see altered evidence fail with a specific mismatch.

Prototype evidence: a synthetic local case rejects the stale step and keeps the reference loader from running. Production outcomes and user impact have not been measured.

CURRENT LIMITATIONS

Local replay has a clear trust boundary.

Evidence and recovery

- **Chain origin:** offline replay checks the address format, not which contract or network produced the bundle.
- **Complete history:** recovery needs saved logs or bundles; the demo has no history indexer.
- **Other trust checks:** past smart-wallet approvals are not replayed; network reads use one RPC source.

Product and meaning

- **Deployment:** Demo Space V2 is synthetic and local; no same-space public deployment is shown.
- **Runtime assurance:** the gate is a local reference adapter, not a production framework integration or formal audit.
- **Real-memory privacy:** extra privacy protection for real memories is not integrated or audited.
- **Memory meaning:** truth, safety, reasoning quality, and memory-to-action causality are out of scope.

What must be proved before real use.

Each milestone has a checkable exit condition; the current demo has not passed these production gates.

01 · CHAIN SOURCE

Trust the source

Verify deployed code and state against a block checked through a trusted source.

02 · HISTORY

Rebuild the same history

Replay saved logs and bundles to the checked registry head; stop if records are missing.

03 · REAL RUNTIME

Gate one real loader

Prove current, approved state proceeds; old, diverged, or unknown state stays blocked.

04 · PEOPLE & PRIVACY

Validate before real memory

Measure unaided reviewer success; test privacy-key backup and recovery.

The question stays simple: is this backup an authorized continuation of the recorded history?

Reproduce locally: `cargo xtask reproduce`

© 2026 MemoryLineage contributors · Open-source code under the MIT License · Third-party notices are in the repository.